Escuela de Ciencias de la Computación
Parcial 2026-I
CC4P1 Programación Concurrente y Distribuida

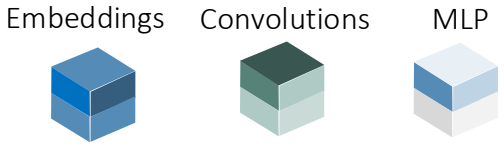
Open AI Cubes es un concepto arquitectónico para sistemas de Inteligencia Artificial modulares, donde cada "cubo" representa una capacidad de IA en una dimensión específica: texto, imágenes, entre otras. Estos cubos se despliegan en entornos distribuidos (múltiples nodos) y cooperan en paralelo para procesar información compleja.

Usted deberá diseñar e implementar una versión funcional de Open AI Cubes que integre dos dimensiones fundamentales (texto e imagen), con capacidad de extensión a futuras dimensiones. Todo el sistema debe construirse exclusivamente con el SDK escogido, sin librerías externas

Micro chunk

Un Micro chunk en una micro parte que es concurrentes paralelo y distribuido que será usado posteriormente por los módulos principales.

- Embeddings
- Convolutions
- MLP (Multilayer Perceptrons)
- Y otros

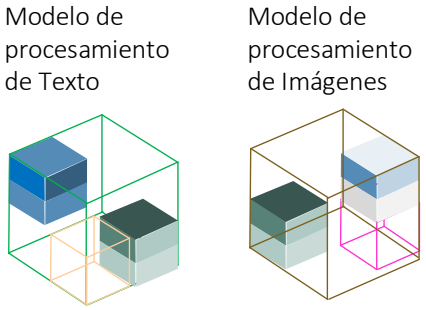


Módulos

Se deben implementar dos módulos principales:

- Modelo de procesamiento de Texto
- Modelo de procesamiento de Imágenes

Cada módulo debe reutilizar los micro chunks que requiera para su consumo también de manera concurrentes paralela y distribuida.

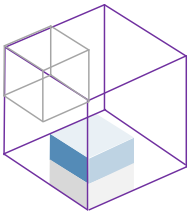


Extensión del Sistema

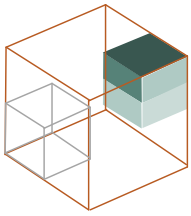
Desarrollar módulos adicionales complementarios para la aplicación:

- Backend: Desarrollar un módulo que consuma los resultados de los modelos de Texto e Imágenes, respondiendo a consultas de los clientes.
- Frontend: Implementar una interfaz de escritorio (desktop) que permita visualizar los resultados al usuario final.

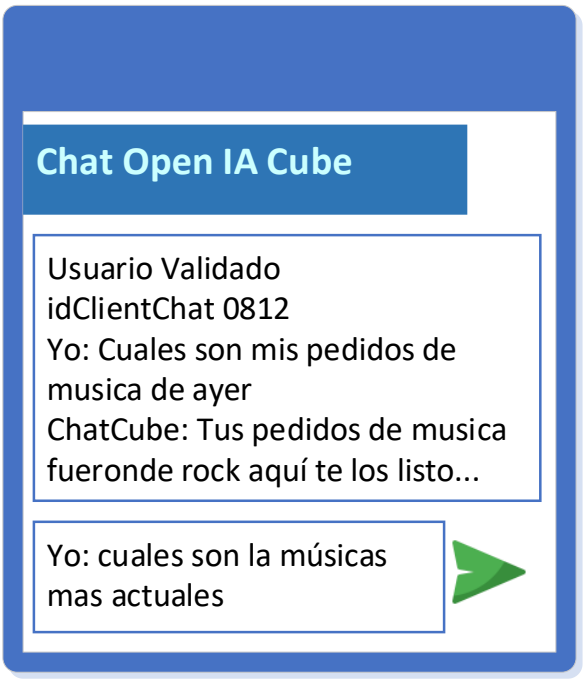
Modulo Backend



Modulo Fontend

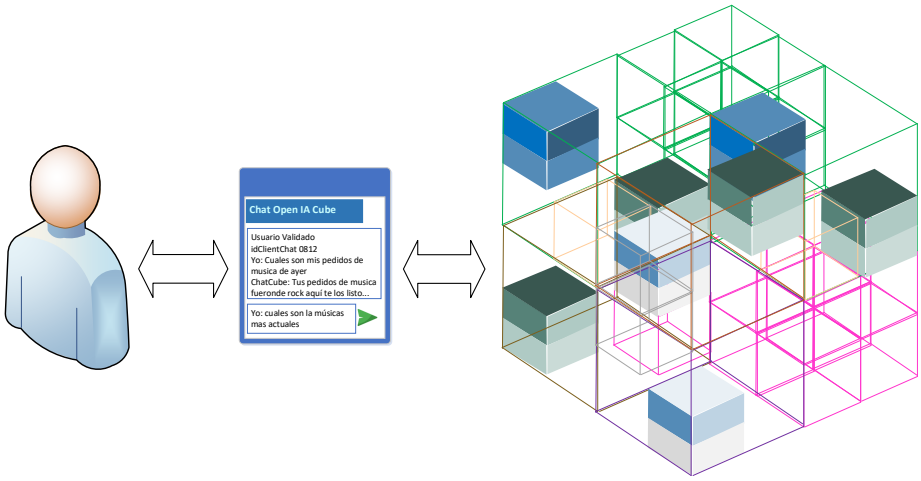


Modulo Fontend



Integración

Integrar todos los módulos (Texto, Imágenes, Backend, Frontend) en un sistema distribuido. Asegurar que el protocolo de comunicación entre nodos permita responder consultas de chat de los clientes de manera eficiente.



Comunicación entre Nodos

Diseñar el protocolo de comunicación entre nodos usando únicamente el SDK del lenguaje elegido (ejemplo: sockets, threads).

Evaluar en varios casos con los tiempos con estos casos.

- **Secuencial** (1 proceso [1 Hilo] secuencial)
- **Paralelo** (1 procesador con 4 núcleos físicos misma maquina [4 hilos])
- **Distribuido** (4 máquinas, 1 núcleo [1 hilo] c/u)
- **Paralelo Distribuido** (2 máquinas × 4 núcleos físicos c/u [4 hilos])

Donde:

LP = lenguaje de programación.

Desarrollo de los Cliente y el Servidor Central (LP1) y Nodos "n" (LP2), donde se tienen que cumplir con los siguientes casos:

- Caso 1 donde LP1 = LP2 (ejem se **ejecute como mínimo jdk 8) (todo el sistema distribuido paralelo funciona con java).**
- **Caso 2 donde LP1 <> LP2, donde los(s) Nodo(s) está en otro lenguaje que no sea Java.**
- **Caso 3 los que tienen grupo de tres o más agregar otros lenguajes de programación en Nodo(s) adicional(es) (ejem LP3, LP4, etc.) se toma en cuenta la cantidad de lenguajes de programación.**

Desarrollar en un cluster de datos para desplegar el programa.

Desarrollando el servidor en internet o en la nube (opcional).

Describir la arquitectura diseñada.

Describir el diagrama de protocolo.

No usar websocket, no usar socket io, no usar algún middleware, no usar otras librerías de conexión u otros protocolos ya trabajados por un tercero.

Usar las Apis, librerías o sdk estándares del lenguaje de programación base.

Explicar el Desarrollo del programa.

Subir en Univirtual.

Se consultará una pregunta en su cluster o en otro cluster de clase.

Hacer pruebas donde los clientes puedan enviar cientos de transacciones regulando con un delay aleatorio entre las transacciones, para saturar el sistema y ver sus límites.

Hacer notas de recordatorio para su exposición.

Comprimido consta

- Subir códigos fuentes de en extensión en el LP1 y LP2 (o LP3), según sea el caso.
- PDF Informe.
- PDF Presentación.
- Evaluar el desempeño con el mayor número de nodos en red con graficas.

Parte 1